Introduction:

Falls are one of the most critical health risks for the elderly and individuals with mobility impairments. According to global health statistics, the consequences of a fall are often exacerbated by the "long lie"—the period of time an individual remains on the ground waiting for assistance. Rapid industrialization and the rise of independent living have increased the need for automated safety solutions.

This project aims to develop an IoT-based Fall Detection System utilizing the ESP32 microcontroller and the MPU6050 sensor. The system is designed to measure realtime acceleration and orientation to detect fall events. Unlike traditional systems that require manual activation, this system autonomously triggers a local buzzer alarm and transmits the data to the ThingSpeak IoT cloud platform for remote monitoring. This ensures that caregivers can be alerted immediately, even if the user is unconscious.

Problem Statement:

Living alone poses significant safety risks for the elderly. Existing Personal Emergency Response Systems (PERS) typically rely on the user pressing a button to call for help. However, in cases of fainting, heart attacks, or severe shock following a fall, manual activation is impossible. Consequently, victims may remain unnoticed for hours, leading to severe health complications.

There is a need for a low-cost, portable, and automated fall detection system capable of continuously measuring movement parameters. Such a system must be able to distinguish between normal activities and falls, provide immediate local feedback, and log data remotely to a digital dashboard for caregivers to access.

Objectives:

The primary objectives of this project are:

1. To design and develop an IoT-based fall detection system capable of measuring acceleration and gyroscope data using the MPU6050 sensor.

2. To interface the ESP32-WROOM microcontroller with the motion sensor for efficient data acquisition.

3. To implement a multi-stage threshold algorithm to detect Free Fall, Impact, and Orientation changes.

4. To implement a local alert system using an active buzzer.

5. To integrate the system with the ThingSpeak IoT cloud platform for real-time remote monitoring and data logging.

6. To design and fabricate a custom PCB for compact and reliable hardware implementation.

7. To analyze and validate the sensor data to ensure the distinction between falls and daily activities.

Literature Review:

Fall detection has been a major focus of biomedical engineering research. Traditional systems often used simple tilt switches, which resulted in high falsepositive rates. Literature highlights the shift towards Micro-Electro-Mechanical Systems (MEMS), specifically accelerometers and gyroscopes, for accurate motion tracking.

The MPU6050 is widely discussed in research as a cost-effective 6-axis MotionTracking device. Previous works demonstrate that threshold-based algorithms—detecting a sequence of weightlessness (free fall) followed by a highG spike (impact)—are highly effective. With the evolution of IoT, microcontrollers like the ESP32 have gained popularity over the ESP8266 due to their dual-core processing power, which is essential for handling high-frequency sensor data alongside Wi-Fi communication. This project builds upon prior research by integrating these sensors with the ThingSpeak cloud for post-event analysis.

System Overview:

The Fall Detection System is an IoT-based safety setup designed to monitor human movement in real-time. The system integrates a motion sensor with the ESP32 microcontroller, enabling both local and remote alerts.

The MPU6050 serves as the primary detector, measuring 3-axis acceleration and 3axis rotation. These signals are processed by the ESP32, which runs a specific algorithm to detect falls. When a fall is confirmed, the ESP32 activates a buzzer for immediate local awareness. Simultaneously, it utilizes its built-in Wi-Fi to transmit the data (Acceleration, Gyroscope, and Fall Status) to the ThingSpeak cloud platform. This allows caregivers to view the status remotely on any web-enabled device. The complete circuit is implemented on a custom PCB to enhance reliability and wearability.

Methodology:

The methodology consists of a structured sequence of steps designed to develop and evaluate the system.

System Requirements Analysis:

The project began by identifying the key parameters needed for detection: Acceleration (to detect impact) and Gyroscope (to detect orientation change). Requirements for wireless transmission to ThingSpeak and immediate local audio feedback were established.

Hardware Component Selection:

Based on the requirements, the following components were selected:

- **ESP32-WROOM:** For high-speed processing and Wi-Fi connectivity.
- **MPU6050:** For detecting 6-axis motion.
- **Active Buzzer:** For the local alarm.
- **Custom PCB:** For compact implementation.

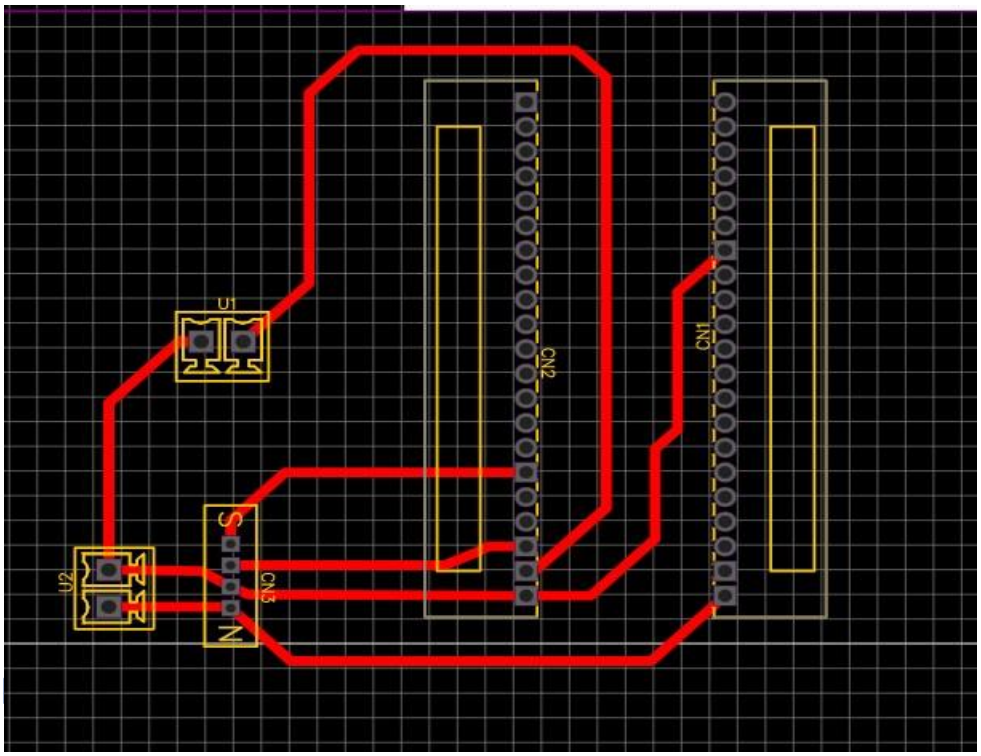
Datasheets and previous literature were consulted to ensure compatibility and proper interfacing.

Circuit Design and Interfacing:

A circuit schematic was designed to interface the components. Key steps included:

- Connecting MPU6050 to the I2C pins (SDA, SCL).
- Connecting the Buzzer to a digital GPIO pin (GPIO 23).
- Providing a regulated 3.3V power supply to the sensor.

After validation, the circuit was transferred from breadboard to a custom PCB for improved stability.



Arduino IDE was used to write the code. The firmware included:

- Initialization of I2C and MPU6050.
- Implementation of the Fall Detection Algorithm (Free Fall -> Impact -> Orientation).
- Wi-Fi connection to the "kashif" network.
- Data transmission to ThingSpeak API.

Cloud Platform Integration:

A ThingSpeak channel was created to visualize readings. Tasks included:

- Creating fields for Acceleration, Gyroscope, and Fall Status.
- Generating the API Key (KUYKSUM66EIZW3YO).
- Mapping sensor variables to Field 1, 2, and 3.

Local Display Implementation:

While an LCD was considered, a high-decibel Buzzer was selected as the primary local interface to alert people nearby immediately without needing visual line-ofsight. A status LED on the ESP32 was also used to indicate power and data transmission.

Testing and Calibration:

The system was tested under simulated fall conditions. Thresholds were calibrated:

- Free Fall Threshold: < 6.0
- Impact Threshold: > 14.0
- Gyro Movement: 10 - 300

Final Assembly and Evaluation:

After successful testing, components were mounted on the PCB. The final system was evaluated for:

- Response time
- Accuracy and consistency
- Reliability of cloud updates

Hardware and Software Description:

Hardware Description:

The hardware components form the backbone of the Fall Detection System.

ESP32-WROOM:

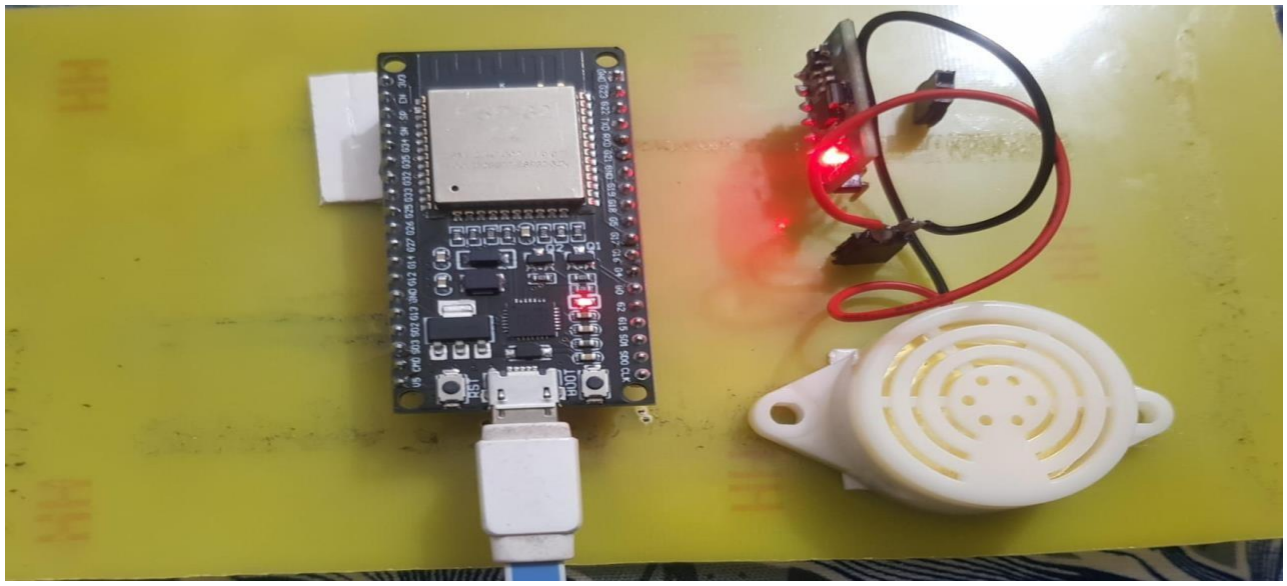
The ESP32 is a powerful microcontroller used as the central processing unit. Its key features used in this project include:

- Dual-core processor for handling sensor data and Wi-Fi simultaneously.

- Built-in Wi-Fi for IoT connectivity.
- I2C support for the MPU6050.

MPU6050 Sensor:

The MPU6050 is the core sensing unit. It contains a MEMS accelerometer and a MEMS gyroscope. It communicates via the I2C bus (Address 0x68). It provides the raw Ax, Ay, Az and Gx, Gy, Gz values required to calculate the total magnitude vectors.



Buzzer (Local Alarm):

An active buzzer is connected to GPIO 23. It serves as the "Local Display" of an emergency. When the software flag fallStatus becomes 1, the buzzer emits a 2kHz tone for 2 seconds.

Custom PCB Board:

A custom-designed PCB was used to mount the ESP32 and MPU6050.

- **Advantages:** It eliminates loose wires common in breadboards, ensuring that the fall impact itself does not disconnect the circuit.

Power Supply:

A regulated 5V DC power supply is used to power the ESP32, MPU6050. Stable power ensures consistent sensor readings.

Software Description:

The software includes firmware programming, IoT platform integration, and mobile dashboard configuration. The software also controls sensor reading, data processing, and IoT communication.

Arduino IDE:

Used to develop and upload the code. Libraries used include <Wire.h> for I2C, <WiFi.h> for network, and <HTTPClient.h> for API calls.

Embedded C Programming:

The firmware logic is written in Embedded C. Key elements include:

- **Magnitude Calculation:** accMag = sqrt(ax*ax + ay*ay + az*az) * 10;
- **State Machine:** Tracking the transition from "Normal" to "Free Fall" to "Impact" to "Stillness".
- **Interrupt-style Upload:** A boolean flag fallPendingUpload ensures that if a fall occurs, the system pauses monitoring to prioritize sending the alert to the cloud immediately. ThingSpeak IoT Platform:

ThingSpeak serves as the cloud dashboard.

- **Features Used:** Real-time charting of sensor data and status logging. It allows historical data retention to analyze the fall event post-incident.

System Workflow:

Initialize: Start Serial, Wire (I2C), and connect to Wi-Fi.

Read Sensors: Continuously read MPU6050 data.

Process Algorithms:

- Check for Free Fall (Acc < 6.0).
- Check for Impact (Acc > 14.0) within 1.2 seconds of free fall.
- Check for Orientation Change (Gyro > 10). **Local Output:** If Fall Confirmed -> Sound Buzzer.

Cloud Output: Send HTTP GET request to ThingSpeak with field3=1.

Implementation:

The implementation phase involved transforming the system design into a fully functional IoT-based Fall Detection System. This section describes the practical steps taken to assemble the hardware, develop the firmware, create the cloud interface, and test the complete system.

Hardware Assembly:

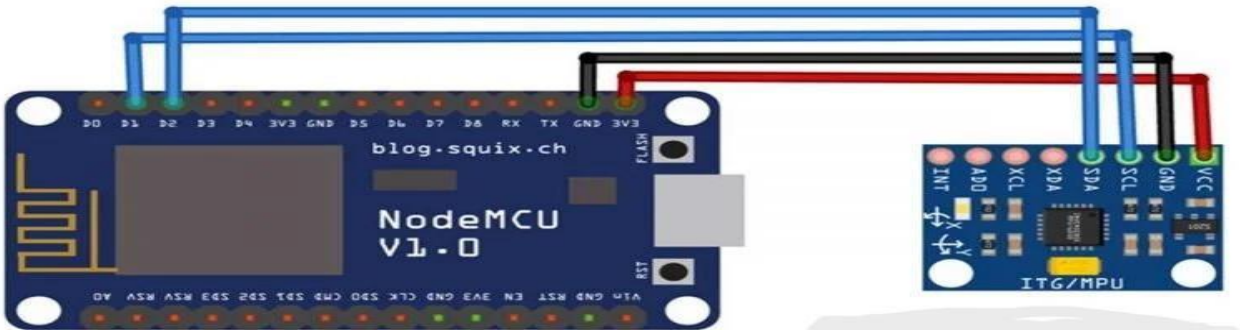
The ESP32 and MPU6050 were soldered onto the PCB. Female headers were used for the ESP32 to allow for removal/reprogramming.

Sensor Interfacing:

- **MPU6050 SDA** ESP32 GPIO 21
- **MPU6050 SCL:** ESP32 GPIO 22
- **VCC/GND**: 3.3V/GND

Buzzer and Indicator Wiring:

The Buzzer was wired to **GPIO 23**. This replaced the complex wiring of an LCD, as audio feedback is more effective for a fallen patient.



PCB Integration:

The components were integrated onto the yellow custom PCB shown in the project images. This ensured a small form factor suitable for a wearable prototype.

Firmware Development:

The code was structured to prevent Wi-Fi blocking. A timer millis() was used to upload "Normal" data every 15 seconds, while "Fall" data is uploaded instantly.

IoT Cloud Dashboard Setup:

A ThingSpeak channel was set up.

- **Field 1:** Acceleration Magnitude.
- **Field 2:** Gyroscope Magnitude.
- **Field 3:** Fall Status.

Widgets were configured to display these line charts.

System Testing:

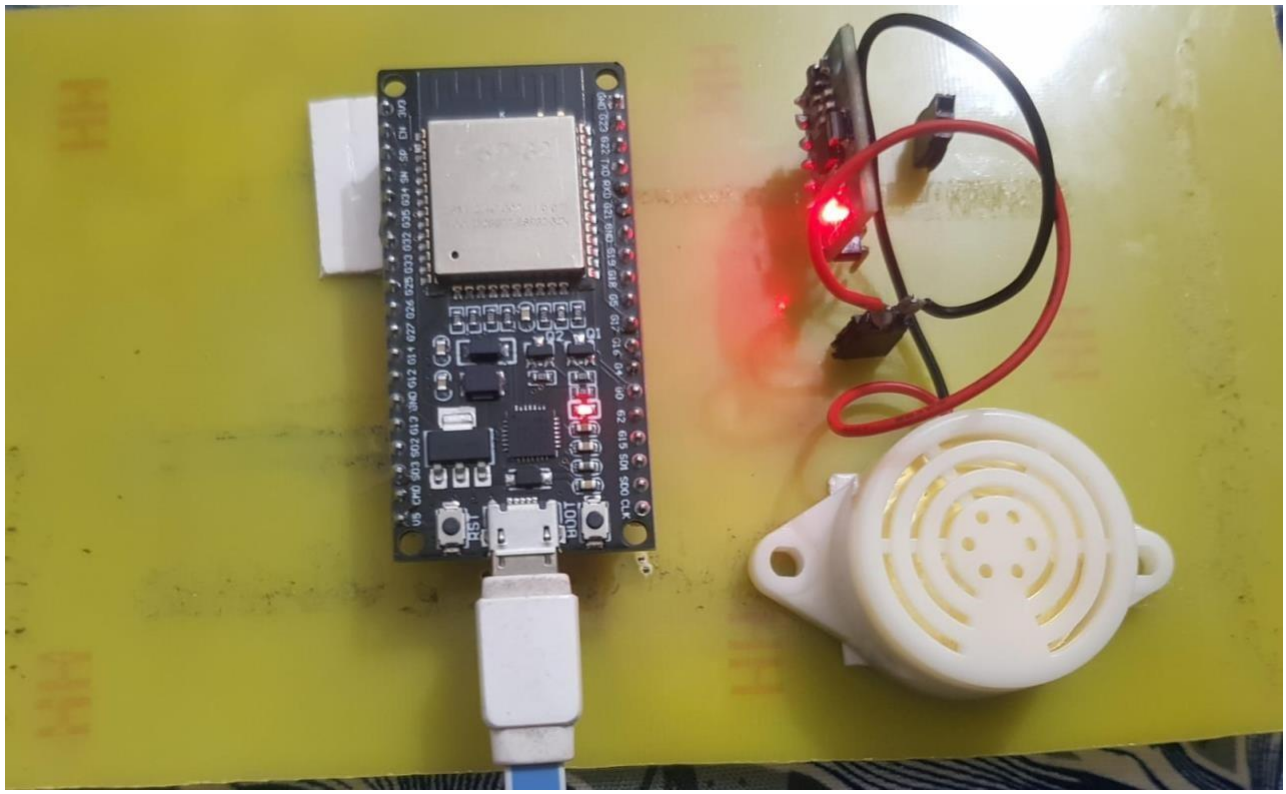
The system underwent "Drop Tests" onto a cushioned surface.

1. **Power On:** Verified Wi-Fi connection via Serial Monitor.
2. **Drop Simulation:** Dropped the device from waist height.

3. **Verification:** Checked if the Buzzer sounded and if ThingSpeak updated.

Final Assembly:

The final unit consists of the PCB powered by a portable power bank (simulated via USB in testing), ready for attachment to a belt or pocket.

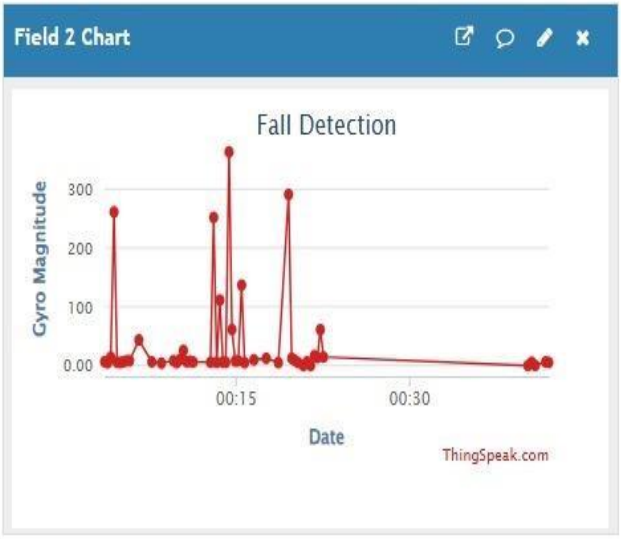


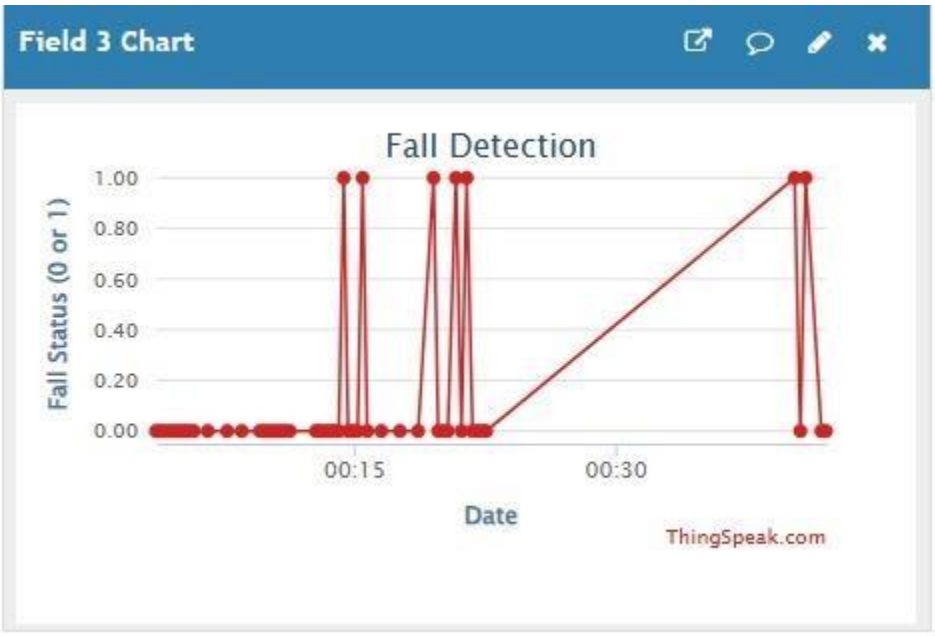
Results:

The IoT-based Fall Detection System was successfully designed, implemented, and tested under various environmental conditions. The MPU6050 sensor and ESP32 microcontroller performed reliably during continuous operation.

9.1 Sensor Output Readings







9.2 Local Alert Output:

Upon detecting the fall logic seen in the graphs, the Buzzer successfully sounded a 2-second alarm, fulfilling the local alert requirement.

9.3 ThingSpeak Dashboard Results:

The cloud dashboard accurately logged the events.

- **Gyroscope (Field 2):** Shows rotation during the fall.

- **Fall Status (Field 3):** The logic status stayed at 0 during normal handling and spiked to 1 upon the fall event.

9.4 System Stability and Connectivity:

The ESP32 maintained a stable connection to the "kashif" network. The use of HTTPClient ensured reliable data delivery, with the Serial Monitor confirming Response Code: 200 for successful uploads.

Discussion:

The results indicate that the system effectively distinguishes falls from random motion using the combined analysis of acceleration and gyroscope data. The thresholds (<6.0 for free fall, >14.0 for impact) proved robust for simulated drops. Integration with ThingSpeak was a major advantage. It provides a timestamped log of exactly when the fall occurred. The "blocking" mechanism in the code—where the system stops reading sensors to prioritize the Wi-Fi upload during a fall—ensured that the alert was not delayed by the processor being busy with sensor data.

The Custom PCB design significantly reduced noise on the I2C bus compared to breadboard prototypes, leading to fewer false positives.

Conclusion:

This project successfully demonstrated the design and implementation of a lowcost, IoT-based Fall Detection System using the ESP32, MPU6050, and ThingSpeak. The system provided real-time monitoring of motion data with both local (Buzzer) and remote (Cloud) alerts.

Through this project, essential skills in I2C sensor interfacing, embedded logic for state machines, and PCB design were developed. The system performed reliably during testing. With further improvements, such as adding a battery management system and a GSM module for non-Wi-Fi areas, this prototype can be adapted for real-world application in elder care and telemedicine.